

Dear reader,

This folder contains the replication package for our oTree implementation of the slider task, which is designed to faithfully reproduce the original z-Tree version in both functionality and interface. If you use our implementation, please cite the accompanying paper: Pinna, L. and R. Vannozzi (2026): “The slider task: An oTree implementation for studying incentive effects in online experiments,” Manuscript in preparation. To successfully implement the slider task, knowledge of oTree is required. To customize its functionality or design, basic JavaScript and HTML skills may also be needed.

In its current configuration, the task proceeds as follows. Participants first land on a welcome page and click a button to continue. They then complete an unpaid practice round of 120 seconds, followed by 10 incentivized rounds, each separated by 10-second breaks. The piece-rate payment per solved slider is set to 2 cents and displayed during the task. Any of these configurations can be modified, as described below.

1. Running the project

1. Choose a suitable version of Python either from [here](#) or install it directly from your laptop’s store. The app was tested using **Python version 3.13.0**
2. Add Python to your PATH environment variable. You can find instructions for this step [here](#).
3. Change the current working directory by using the “cd” command on the terminal, followed by the path to the folder that contains the oTree project.
4. Install the dependencies by typing in the terminal

```
pip install -r requirements.txt
```

5. To initialize a development server on your laptop, type in the terminal

```
otree devserver
```

6. Open `http://localhost:8000/` in your browser, select the `slider_task` session, and open the session-wide demo link to start the task as a participant.

Note that the development server is for local testing only. For online data collection, you must set up a hosted server. You can find instructions for this step [here](#).

2. Contents of the folder

The package follows the standard layout for oTree projects: it contains `settings.py` and `requirements.txt` at the root, and the `slider_task` folder with the app’s logic and pages. Where helpful, we include explanatory comments directly in the files.

Root files

- `settings.py` : Includes project settings such as the session configuration (named `slider_task`), the currency (USD), the number of participants, and administrative settings. Before any public deployment, replace `SECRET_KEY` with your own value. This file also declares the participant-level variables, which persist across all rounds for a given participant.
- `requirements.txt` : Includes Python dependencies. Only oTree is required to run the project.

Application logic

- `__init__.py` : Defines the player variables, the page logic, and the page sequence. The constants at the top of the file are the parameters you are most likely to change:
 - `NUM_ROUNDS` : Number of paid rounds (default 10)
 - `TIME_FOR_TASK` : Length of each paid round, in seconds (default 120)
 - `TIME_FOR_BREAK` : Break between rounds, in seconds (default 10)
 - `PIECE_RATE` : Payment per correctly placed slider, in cents (default 2)

The file also defines the variables recorded in each round for a given participant.

Pages

- `Welcome.html` : Landing page. It contains the script that records screen size (in CSS pixels) and detects mobile devices.
- `DropMobile.html` : Exclusion message shown to mobile users.
- `BeforePractice.html` : Blank page that starts the practice-round timer. It auto-submits and is not seen by the participant.
- `PracticeRound.html` : The practice-round interface.
- `BeforeRounds.html` : Blank page that starts the timer for each paid round. It auto-submits and is not seen by the participant.
- `Slider.html` : The paid-round interface, which constitutes the core of the package.
- `Break.html` : The between-rounds break page.
- `StudyComplete.html` : The closing page.

3. Data recorded

The main variables stored by the application are the following:

- `score` : number of sliders solved in each paid round.
- `tot_score` : total number of sliders solved across all paid rounds.
- `tot_payment` : total payment for the study.
- `practice_score` : number of sliders solved in the practice round.
- `initial_zoom` : browser device pixel ratio at the beginning of the task.
- `current_zoom` : updated device pixel ratio if browser zoom changes.
- `uses_mobile` : indicator for devices flagged as mobile.

- `screen_height` and `screen_width`: participant screen size in CSS pixels.

4. Interface and functionality of the slider task

`Slider.html` and `PracticeRound.html` contain the JavaScript code that draws the sliders and handles dragging and scoring. The code is divided into numbered sections, each including comments explaining its purpose.

Simple adjustments require no programming knowledge. For example, the number of sliders displayed can be changed by adjusting the `sliderCount` variable near the top of the script, up to a maximum of 48. The screen-scaling rule, which compares the screen against a 1920×1080 reference and shrinks the elements to fit smaller displays, can also be adjusted. More substantial customization requires JavaScript knowledge. The sections that, when modified, are more likely to hinder the interface are marked “DO NOT MODIFY”.

5. Optional toggles

The project includes optional control over participants’ behavior or device used:

Mobile exclusion. By default, participants on a smartphone or tablet are sent to the `DropMobile` page and asked to leave the study. Each participant’s device type is recorded on `Welcome.html` by JavaScript functions that detect touchscreen support, a coarse pointer, and actual touch events. The default filter is restrictive because it excludes any participant meeting any of these criteria. False positives are therefore possible, especially among participants using touchscreen-capable laptops. The filter can be relaxed by reducing the number of criteria, or disabled entirely by commenting out `DropMobile` in the page sequence at the bottom of `__init__.py`.

Screen size. `Welcome.html` also stores the participant’s screen dimensions in CSS pixels. These variables can be used as additional controls, or as the basis for excluding participants whose screen is too small to complete the task effectively. No such filter is enforced by default.

Fixed-zoom enforcement. Increasing browser zoom mid-task can enlarge the sliders and make them easier to position correctly. You can prevent this by uncommenting the indicated line inside the `checkDPRChange()` function in `Slider.html`. When active, an overlay covers the task after any zoom change and disappears only once the participant restores the original zoom level.